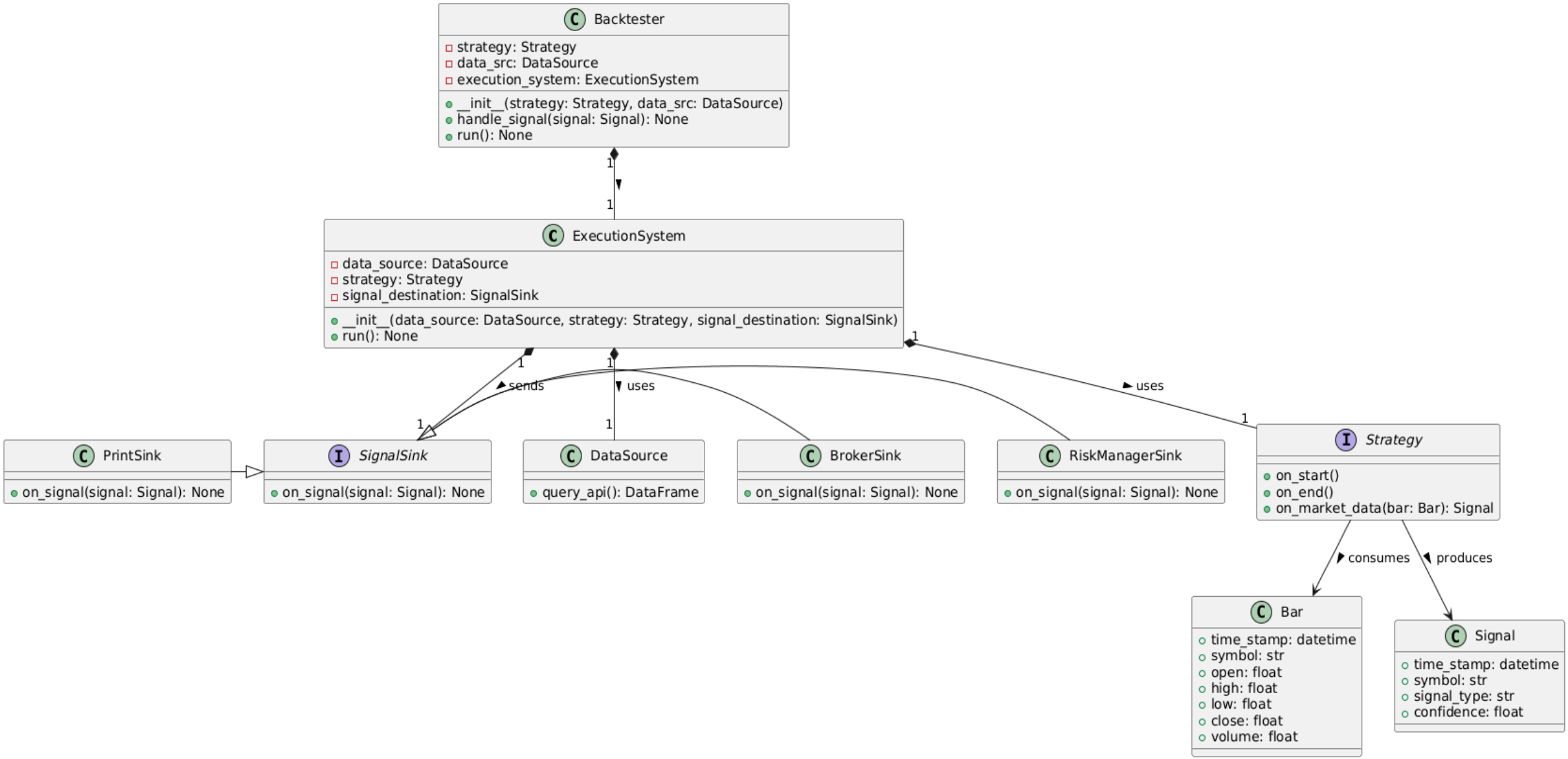
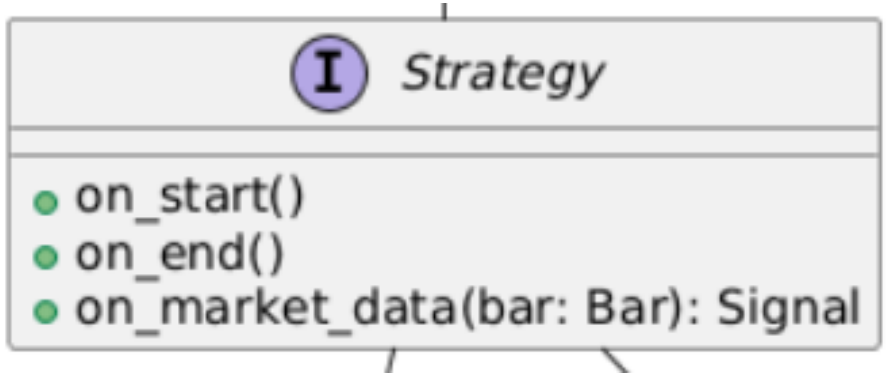
Execution System

Stand Up Notes

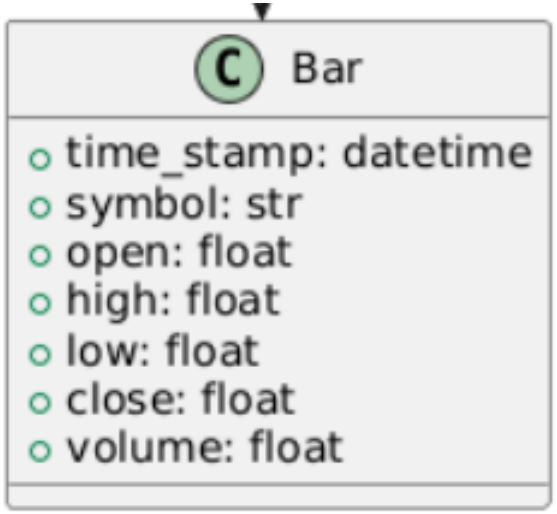

Backtester
- strategy: Strategy
- data_src: DataSource
- execution_system: ExecutionSystem
+ __init__(strategy: Strategy, data_src: DataSource)
+ handle_signal(signal: Signal): None
+ run(): None
1
1
ExecutionSystem
- data_source: DataSource
- strategy: Strategy
- signal_destination: SignalSink
+ __init__(data_source: DataSource, strategy: Strategy, signal_destination: SignalSink)
+ run(): None
1
1
sends
1
1
uses
1
uses
1
PrintSink
+ on_signal(signal: Signal): None
SignalSink
+ on_signal(signal: Signal): None
DataSource
+ query_api(): DataFrame
BrokerSink
+ on_signal(signal: Signal): None
RiskManagerSink
+ on_signal(signal: Signal): None
Strategy
+ on_start()
+ on_end()
+ on_market_data(bar: Bar): Signal
consumes
produces
Bar
+ time_stamp: datetime
+ symbol: str
+ open: float
+ high: float
+ low: float
+ close: float
+ volume: float
Signal
+ time_stamp: datetime
+ symbol: str
+ signal_type: str
+ confidence: float

Interface Design

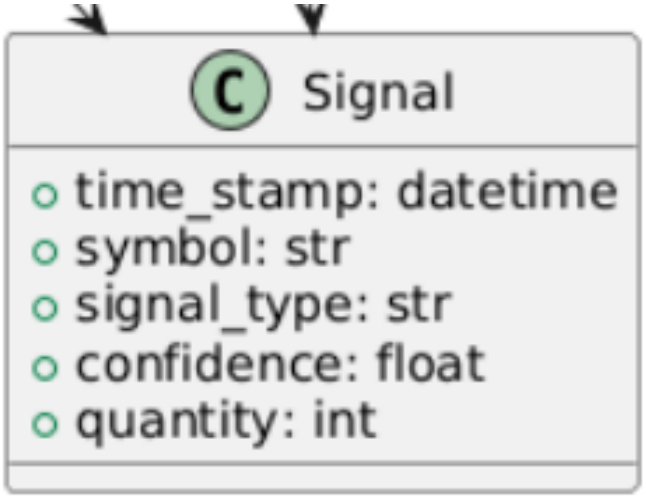


Interface Implementation

```python
class Strategy:
    def on_start(self): ...
    def on_end(self): ...
    def on_market_data(self, bar: Bar) -> Signal: ...
```

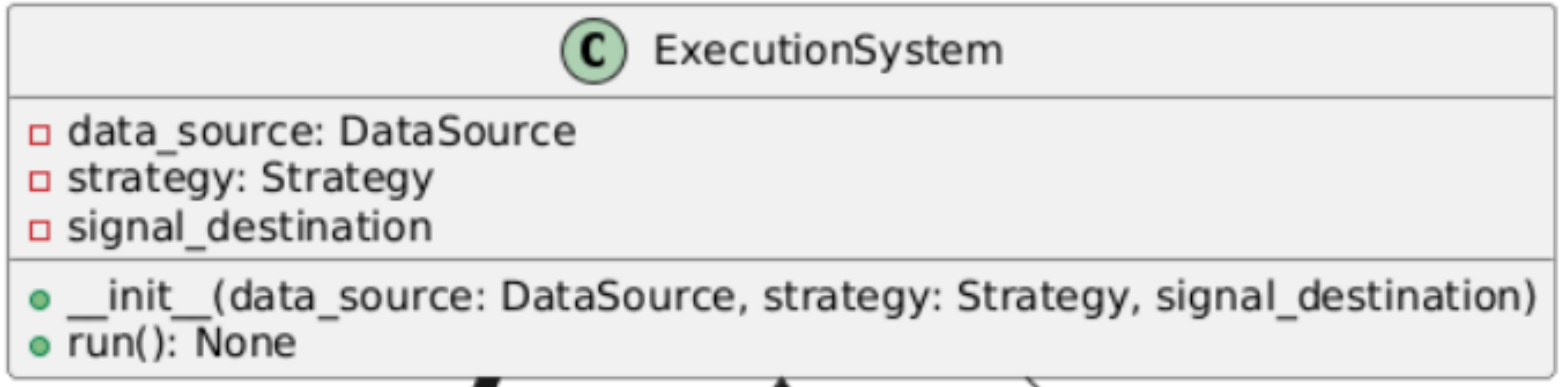



```python
@dataclass(frozen=True)
class Bar:
    time_stamp: datetime
    symbol: str
    open: float
    high: float
    low: float
    close: float
    volume: float
```




```python
@dataclass(frozen=True)
class Signal: # TODO: expand
    time_stamp: datetime
    symbol: str

    signal_type: str
    confidence: float
```




```python
class ExecutionSystem:
    def __init__(
            self,
            data_source: DataSource,
            strategy: Strategy,
            signal_destination=None, # TODO: Define type

    ) -> None:
        self.data_source = data_source
        self.strategy = strategy
        self.signal_destination = signal_destination # Backtester / Broker

    async def run(self) -> None:
        self.strategy.on_start()

        async for bar in self.data_source.__aiter__():
            self.strategy.on_market_data(bar)

        self.strategy.on_end()
```

```python
class DataSource(Protocol):
    def __init__(self, symbol: str = "^GSPC"):
        self.symbol = symbol

    async def __aiter__(self) -> AsyncIterable[Bar]:
        year = 2020
        month = 1
        start = f"{year}-{month:02d}-01"
        end = f"{year}-{month:02d}-28"

        print(f"Fetching {self.symbol} data from {start} to {end}...")

        data = yf.download(self.symbol, start=start, end=end, progress=False, auto_adjust=True)

        for ts, row in data.iterrows():
            bar = Bar(
                time_stamp=ts.to_pydatetime(),
                symbol=self.symbol,
                open=float(row["Open"].item()),
                high=float(row["High"].item()),
                low=float(row["Low"].item()),
                close=float(row["Close"].item()),
                volume=float(row["Volume"].item()),
            )
            yield bar
            await asyncio.sleep(0.1)
```

*this is just a simple abstraction, BUT, the Execution system relies on the fact that it will be able to call an async method that returns an AsyncIterable[Bar]

*this part needs to be discussed with Data team

Example main

```python
if __name__ == "__main__":
    print("Example Implementation: ")

    # startegy receives market data and produces buy/sell signal
    strategy = ExampleStrategy()

    # this is a sample implementation of what the data source could look like
    # DataSource is just a yfinance wrapper with hard coded variables
    # it provides historical market data as an async iterable of Bar objects
    data = DataSource()

    # ExecutionSystem coordinates the backtesting loop:
    #    - fetches historical data
    #    - feeds it into the startegy
    #    - TODO: redirect the signals to a) RiskManger or b) Backtester
    execution = ExecutionSystem(data_source=data, strategy=strategy)

    # ExecutionSystem.run() is an async routine so it must be run
    # with asyincio.run() to start the event loop
    asyncio.run(execution.run())
```

```python
class ExampleStrategy(Strategy):
    '''
    A simple example strat that generates signals based on the previous closing price:
        - if current bar closes higher than previous -> SELL signal
        - if current bar closes lower than previous -> BUY signal
    * this is just a minimal implementation to exemplify how the interface would be used
    '''
    def __init__(self):
        self.prev_bar: Bar | None = None

    def on_start(self):
        print("Strategy started.")
        self.prev_bar = None

    def on_end(self):
        print("Strategy finished.")

    def on_market_data(self, bar: Bar) -> Signal | None:
        if self.prev_bar is None:
            self.prev_bar = bar
            return None

        signal_type = None
        self.prev_bar = bar

        if bar.close > self.prev_bar.close:
            signal_type = "SELL"
        elif bar.close < self.prev_bar.close:
            signal_type = "BUY"

        print("Signal Type: ", signal_type, ", Timestamp: ", bar.time_stamp, ", Symbol: ", bar.symbol, "Confidence: ", 1)

        if signal_type is None:
            return None

        return Signal(time_stamp=datetime.now(), symbol=bar.symbol, signal_type=signal_type, confidence=1.0)
```

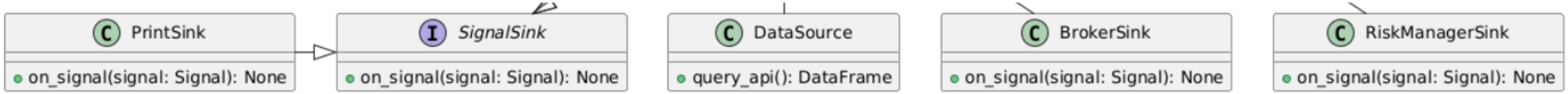
Sample Strategy

Usage

```
$ cd GhosalQuant
$ python3 src/example.py
```

```
Strategy started.
Fetching ^GSPC data from 2020-01-01 to 2020-01-28...
Signal Type:  BUY , Timestamp:  2020-01-03 00:00:00 , Symbol:  ^GSPC Confidence:  1
Signal Type:  SELL , Timestamp:  2020-01-06 00:00:00 , Symbol:  ^GSPC Confidence:  1
Signal Type:  BUY , Timestamp:  2020-01-07 00:00:00 , Symbol:  ^GSPC Confidence:  1
Signal Type:  SELL , Timestamp:  2020-01-08 00:00:00 , Symbol:  ^GSPC Confidence:  1
Signal Type:  SELL , Timestamp:  2020-01-09 00:00:00 , Symbol:  ^GSPC Confidence:  1
Signal Type:  BUY , Timestamp:  2020-01-10 00:00:00 , Symbol:  ^GSPC Confidence:  1
Signal Type:  SELL , Timestamp:  2020-01-13 00:00:00 , Symbol:  ^GSPC Confidence:  1
Signal Type:  BUY , Timestamp:  2020-01-14 00:00:00 , Symbol:  ^GSPC Confidence:  1
Signal Type:  SELL , Timestamp:  2020-01-15 00:00:00 , Symbol:  ^GSPC Confidence:  1
Signal Type:  SELL , Timestamp:  2020-01-16 00:00:00 , Symbol:  ^GSPC Confidence:  1
Signal Type:  SELL , Timestamp:  2020-01-17 00:00:00 , Symbol:  ^GSPC Confidence:  1
Signal Type:  BUY , Timestamp:  2020-01-21 00:00:00 , Symbol:  ^GSPC Confidence:  1
Signal Type:  SELL , Timestamp:  2020-01-22 00:00:00 , Symbol:  ^GSPC Confidence:  1
Signal Type:  SELL , Timestamp:  2020-01-23 00:00:00 , Symbol:  ^GSPC Confidence:  1
Signal Type:  BUY , Timestamp:  2020-01-24 00:00:00 , Symbol:  ^GSPC Confidence:  1
Signal Type:  BUY , Timestamp:  2020-01-27 00:00:00 , Symbol:  ^GSPC Confidence:  1
Strategy finished.
```

Output




```python
class SignalSink(Protocol):
    async def on_signal(self, signal: Signal) -> None: ...

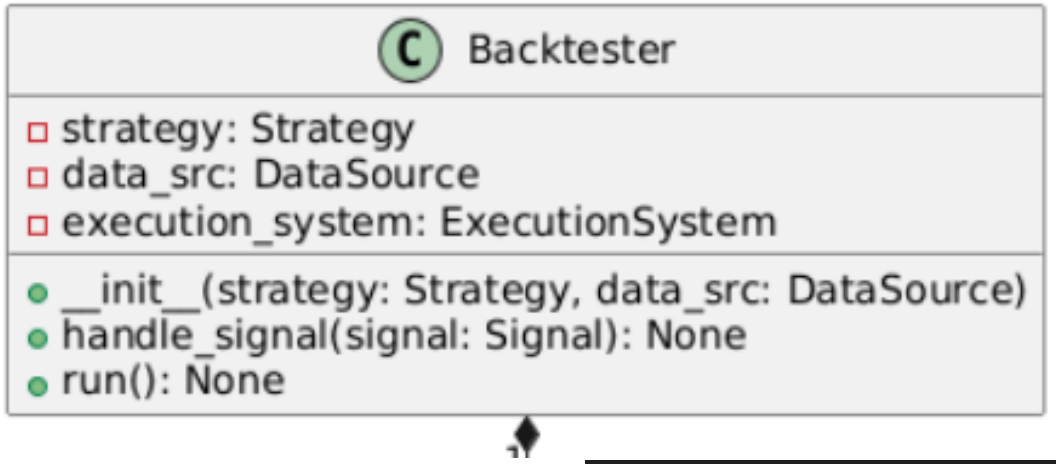
class PrintSink(SignalSink):
    async def on_signal(self, signal: Signal) -> None:
        print(f"Received signal: {signal}")

class BacktesterSink(SignalSink):
    def __init__(self, backtester) -> None:
        self.backtester = backtester

    async def on_signal(self, signal: Signal) -> None:
        await self.backtester.handle_signal(signal)

class BrokerSink(SignalSink):
    async def on_signal(self, signal: Signal) -> None:
        raise NotImplementedError("BrokerSink is not implemented yet.")

class RiskManagerSink(SignalSink):
    async def on_signal(self, signal: Signal) -> None:
        raise NotImplementedError("RiskManager is not implemented yet.")
```




```python
class Backtester:
    def __init__(self, strategy: Strategy, data_src: DataSource):
        self.strategy = strategy
        self.data_src = data_src
        self.execution_system = ExecutionSystem(
            data_src = self.data_src,
            strategy = self.strategy,
            signal_destination = BacktesterSink(self)
        )

    async def handle_signal(self, signal) -> None: ...

    def run(self): ...
```

Assumptions made

- The Execution class is instantiated in the main with Strategy and Data Source parameters passed.

- The method that gets the data in the DataSource is an async that returns AsyncIterable[Bar] to avoid the system having to wait for new data to be fetched.

Further questions

- Concretise the overall architecture in more detail, relationships between components, and how they communicate/interact (apis)

- Need for a Risk Manager System, in order to calculate quantities based on confidence, keep track of open positions (maybe?), etc.